

El Pañol: un sistema de préstamo de equipos

Acabas de ver en vivo cómo se arma **El Recinto**. Ahora te toca a ti, con otro sistema y contra la misma API real. Mismo esqueleto técnico —**módulos de Vuex, Vue Router y axios**— con un negocio distinto y reglas propias.

Este documento **no trae la solución**. Trae el encargo, la documentación de la API y los criterios con los que se va a revisar tu trabajo. El código lo escribes tú.

EN VIVO

El encargo

El **pañol** de la institución presta equipos: notebooks, proyectores, routers, multímetros. Hoy funciona con un cuaderno y un lápiz. Cuando alguien pregunta "¿quedan notebooks?", la respuesta es revisar hoja por hoja quién se llevó qué y quién todavía no lo trae.

Te piden una aplicación para el mesón. Tiene que dejar ver el inventario con las unidades realmente disponibles, registrar préstamos nuevos, marcar entregas y devoluciones, y mostrarle al encargado un panel con lo que está fuera y lo que va atrasado.

El backend ya existe y ya funciona. Tú construyes el frontend completo.

CUIDADO

Lo primero: esto NO es el mismo problema que el Recinto. Léelo antes de abrir el editor.

La estructura es idéntica —un catálogo del que depende un CRUD— pero la regla de negocio es de otra naturaleza, y si intentas copiar el código de la clase te vas a estrellar:

En el Recinto el conflicto era una **colisión exacta**: esa cancha, ese día, esa hora. Una pregunta de sí o no.

Acá el conflicto es de **capacidad**: no importa cuándo, importa *cuántas unidades quedan* sumando todos los préstamos que siguen fuera. Una pregunta de cuántos, no de sí o no.

Lo que sí se transfiere es la **idea**: hay una regla que sólo el servidor puede evaluar, responde `409`, y tu interfaz tiene que saber mostrarla distinto de un error de campo.

MATIZ

Qué se evalúa y qué no.

Se evalúa **dónde pusiste cada cosa**: si axios vive sólo en las actions, si los módulos tienen sentido, si el estado del formulario está donde corresponde, si distingues un 422 de un 409.

No se evalúa el diseño. Más abajo tienes una hoja de estilos completa para copiar y pegar: úsala y no pierdas ni un minuto maquetando. Si tu pantalla se ve fea pero el estado está bien manejado, está aprobada.

RUTA DE TRABAJO



De dónde partes

· proyecto, dependencias y la API que ya existe

1

Requisito 1 — Las cuatro pantallas y su navegación

· router

2

Requisito 2 — El inventario: primer módulo

· axios en una action

3

Requisito 3 — La planilla de préstamos: segundo módulo

· CRUD y filtros

4

Requisito 4 — El formulario en Vuex

· 422 contra 409

5

Requisito 5 — El panel del encargado

· getters y /resumen

✓ **Cómo se revisa · Si terminas antes · Errores que te van a pasar**

◆ De dónde partes

Antes de escribir una línea de Vue. Levanta el backend, crea el proyecto y **recorre la API en Postman**. Ese último paso no es opcional.

1 Paso 1 — El backend

> Terminal • una pestaña sólo para esto

```
cd node-backend-edu
npm install
npm run dev

# 🚀 Servidor corriendo en http://localhost:3001
```

2 Paso 2 — El proyecto

> Terminal • otra pestaña

```
npm create vite@latest panol -- --template vue
cd panol
npm install
npm install vuex@4 vue-router@4 axios
npm run dev
```

3 Paso 3 — Recorre la API antes de programar

CLAVE

Abre la colección de Postman y dispara estas cinco peticiones. En ese orden.

Vas a escribir código que consume estas respuestas. Si no sabes qué forma tienen, vas a programar a ciegas y después vas a culpar al framework.

› Terminal • o su equivalente en Postman

```
# 1. El catálogo. Mira el campo `disponibles`: NO está guardado, se calcula.
curl -s http://localhost:3001/api/libre/equipos

# 2. Un préstamo. Mira `atrasado` y `diasAtraso`: tampoco están guardados.
curl -s http://localhost:3001/api/libre/prestamos/4

# 3. La planilla. ¿Es un arreglo o un objeto? Averígualo antes de escribir el v-fo
curl -s "http://localhost:3001/api/libre/prestamos?porPagina=3"

# 4. Las cifras del pañol.
curl -s http://localhost:3001/api/libre/prestamos/resumen

# 5. Pide una cámara. Hay 3 y Las 3 están fuera. Mira bien lo que responde.
curl -s -X POST http://localhost:3001/api/libre/prestamos \
  -H 'Content-Type: application/json' \
  -d '{"equipoId":3,"solicitante":"Curso de Cine","area":"Audiovisual",
    "cantidad":1,"fechaRetiro":"2026-08-10","fechaDevolucion":"2026-08-15"}'
```

CUIDADO

Esa quinta petición es el ejercicio entero en una línea.

Todos los campos son válidos. El equipo existe, está operativo, las fechas son reales y están en orden, la cantidad es razonable. **Y el servidor la rechaza igual**, con un `409`, porque no queda ninguna cámara.

Ninguna validación que escribas en el navegador puede prevenir eso. Depende de lo que otras personas pidieron antes que tú. Guárdate esa idea: la vas a necesitar en el Requisito 4.

4 La API que tienes disponible

Todo cuelga de `http://localhost:3001/api/libre`. **No pide token**: hoy el tema es el manejo del estado, no la autenticación.

EQUIPOS — CATÁLOGO DE SÓLO LECTURA

MÉTODO Y RUTA	PARA QUÉ	RESPONDE
GET /equipos	Inventario completo	Arreglo directo
GET /equipos/categorias	Opciones del selector	{ categorias: [{ valor, etiqueta }] }
GET /equipos/:id	Un equipo	Objeto directo

Filtros de GET /equipos: categoria, operativo (true/false), conStock (true).

⇒ Forma de un equipo

```
{
  "id": 1,
  "nombre": "Notebook Lenovo ThinkPad",
  "categoria": "computacion",
  "marca": "Lenovo",
  "stockTotal": 12,           // unidades que EXISTEN
  "valorUnitario": 620000,
  "operativo": true,         // si es false, no se puede prestar

  // Estos DOS los calcula el servidor en cada consulta.
  // No están guardados en ninguna parte.
  "comprometidas": 9,         // unidades fuera del pañol ahora mismo
  "disponibles": 3           // stockTotal - comprometidas
}
```

PRÉSTAMOS — EL CRUD

MÉTODO Y RUTA	PARA QUÉ	RESPONDE
GET /prestamos	Planilla con filtros y paginación	{ datos, meta }
GET /prestamos/opciones	Estados y categorías	{ estados, estadosVivos, categorias }
GET /prestamos/resumen	Cifras del pañol	Objeto con agregaciones

MÉTODO Y RUTA	PARA QUÉ	RESPONDE
GET /prestamos/:id	Un préstamo	Objeto directo
POST /prestamos	Registrar	201 + cabecera Location
PUT /prestamos/:id	Reemplazo total	El préstamo
PATCH /prestamos/:id	Entregar o devolver	El préstamo
DELETE /prestamos/:id	Borrar	{ message, id }

↔ Forma de un préstamo

```
{
  "id": 4,
  "codigo": "PR-0004",
  "equipoId": 3,
  "solicitante": "Marcelo Vera",
  "area": "Registro Audiovisual",
  "cantidad": 2,
  "fechaRetiro": "2026-07-20",
  "fechaDevolucion": "2026-07-31",
  "estado": "entregado",           // pendiente | entregado | devuelto
  "observacion": "Cobertura de la ceremonia.",
  "creadoEn": "...",
  "actualizadoEn": "...",

  // Todo esto lo resuelve el servidor. Úsalo, no lo recalculas.
  "equipoNombre": "Cámara de video",
  "equipoCategoria": "audiovisual",
  "atrasado": true,               // entregado y con la fecha vencida
  "diasAtraso": 4,
  "valorComprometido": 1780000   // valorUnitario x cantidad
}
```

Filtros de GET /prestamos:

PARÁMETRO	VALORES	POR DEFECTO
estado	pendiente · entregado · devuelto	—
equipoId	Id de un equipo existente	—
categoria	Una de las cinco categorías	—
atrasados	true · false	—
buscar	Solicitante, área, código o equipo	—
orden	recientes · solicitante · devolucion · cantidad	recientes
pagina · porPagina	Números. porPagina máximo 50	1 · 6

LOS TRES ESTADOS Y QUÉ SIGNIFICAN PARA EL STOCK

ESTADO	QUÉ PASÓ	¿OCUPA UNIDADES?
pendiente	Reservado, todavía no lo retiran	Sí
entregado	Está en manos del solicitante	Sí
devuelto	Volvió al pañol	No — libera el stock

VALIDACIÓN — RESPONDE 422 CON DETALLE POR CAMPO

CAMPO	REGLA
equipoId	Obligatorio. El equipo debe existir y estar operativo
solicitante	Obligatorio, 3 a 60 caracteres
area	Obligatorio, mínimo 3 caracteres
cantidad	Entero entre 1 y 20

CAMPO	REGLA
<code>fechaRetiro</code>	Obligatorio, <code>AAAA-MM-DD</code> y fecha real
<code>fechaDevolucion</code>	Obligatorio, y nunca anterior al retiro
<code>observacion</code>	Opcional, máximo 200 caracteres

CONFLICTOS — RESPONDEN 409

SITUACIÓN	CUERPO DE LA RESPUESTA
No alcanzan las unidades	Trae un objeto <code>stock</code> con <code>solicitadas</code> , <code>disponibles</code> , <code>comprometidas</code> y <code>stockTotal</code>
Modificar un préstamo ya devuelto	Trae <code>estadoActual</code>

⇒ El 409 por stock – mira lo que te devuelve

```
{
  "message": "No hay unidades suficientes de Notebook Lenovo ThinkPad. Pediste 5 y
  "stock": {
    "equipo": "Notebook Lenovo ThinkPad",
    "solicitadas": 5,
    "disponibles": 3,
    "comprometidas": 9,
    "stockTotal": 12
  }
}
```

CLAVE

El servidor no sólo te dice que no. Te dice cuántas hay.

Con esos números puedes hacer que tu interfaz sea útil de verdad: en vez de un "error" seco, un mensaje que ofrezca pedir las 3 que sí quedan. Eso está en los criterios de aceptación del Requisito 4.

Crea `src/estilos.css` con esto e impórtalo en `main.js` con `import './estilos.css'`. Las clases están pensadas para el HTML que vas a escribir.

✓ `src/estilos.css` SOLUCIÓN

```
* { box-sizing: border-box; }

body {
  margin: 0;
  background: #f4f6f8;
  color: #1c2530;
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
}

/* ----- Layout ----- */
.barra {
  display: flex; align-items: center; gap: 1.5rem;
  padding: 0 1.5rem; height: 60px;
  background: #1f2d3a; color: #fff;
  position: sticky; top: 0; z-index: 10;
}
.barra h1 { font-size: 1.05rem; margin: 0; white-space: nowrap; }
.menu { display: flex; gap: .5rem; flex: 1; }
.menu a {
  color: #b9c7d4; text-decoration: none; padding: .5rem .9rem;
  border-radius: 8px; font-weight: 600; font-size: .9rem;
}
.menu a:hover { background: rgba(255,255,255,.08); color: #fff; }
.menu a.router-link-active { background: #2f8f66; color: #fff; }
.contenido { max-width: 1100px; margin: 0 auto; padding: 1.75rem 1.5rem 4rem; }

/* ----- piezas ----- */
.tarjeta {
  background: #fff; border: 1px solid #dfe6ec;
  border-radius: 12px; padding: 1.1rem;
}
.grilla {
  display: grid; gap: 1rem;
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
}
.cabecera {
```

```

    display: flex; align-items: flex-start; justify-content: space-between;
    gap: 1rem; margin-bottom: 1.25rem;
}
.sub { margin: 0; color: #64748b; font-size: .88rem; }

.btn {
    background: #2f8f66; color: #fff; border: 0; border-radius: 8px;
    padding: .55rem 1rem; font-weight: 600; font-size: .88rem;
    text-decoration: none; cursor: pointer; white-space: nowrap;
    display: inline-block;
}
.btn:disabled { background: #9bb8ab; cursor: not-allowed; }
.btn--gris { background: #94a3b8; }
.btn--fino { padding: .25rem .55rem; font-size: .78rem; }

.aviso {
    background: #fff; border: 1px solid #dfe6ec; border-radius: 10px;
    padding: 2rem; text-align: center; color: #64748b;
}
.aviso--error { background: #fdeceb; border-color: #f0b7b1; color: #96251a; }

/* ----- tabla ----- */
.tabla-caja {
    background: #fff; border: 1px solid #dfe6ec;
    border-radius: 10px; overflow-x: auto;
}
table { width: 100%; border-collapse: collapse; font-size: .87rem; min-width: 760px; }
th, td { padding: .65rem .85rem; text-align: left; border-bottom: 1px solid #eef2f3; }
th {
    background: #f7f9fb; font-size: .73rem; text-transform: uppercase;
    letter-spacing: .04em; color: #64748b;
}
tbody tr:last-child td { border-bottom: 0; }
tr.ocupada { opacity: .5; pointer-events: none; }
tr.atrasada { background: #fdf5f4; }
td small { display: block; color: #94a3b8; font-size: .76rem; }
.mono { font-family: ui-monospace, Menlo, monospace; font-size: .8rem; color: #64748b; }

/* ----- etiquetas de estado ----- */
.chip {
    font-size: .73rem; font-weight: 700; padding: .18rem .55rem;
    border-radius: 999px; text-transform: capitalize; display: inline-block;
}
.chip--pendiente { background: #fdf3e4; color: #96590c; }
.chip--entregado { background: #e9f1fb; color: #2a6fc9; }

```

```

.chip--devuelto { background: #e9f6f0; color: #2f8f66; }
.chip--atraso   { background: #fdeceb; color: #c0392b; }

/* ----- formulario ----- */
.form { display: flex; flex-direction: column; gap: 1rem; max-width: 640px; }
.fila { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
label { display: flex; flex-direction: column; gap: .35rem; }
label > span { font-size: .82rem; font-weight: 700; color: #475569; }
input, select, textarea {
  padding: .55rem .7rem; border: 1px solid #cfd9e2; border-radius: 8px;
  font-size: .92rem; font-family: inherit; background: #fff;
}
input:focus, select:focus, textarea:focus {
  outline: 2px solid #42b883; outline-offset: 1px; border-color: #42b883;
}
.malo { border-color: #c0392b; background: #fdf5f4; }
.malo-txt { color: #c0392b; font-size: .79rem; font-weight: 600; }

/* La caja del 409. Distinta a propósito de los errores de campo. */
.conflicto {
  background: #fdf3e4; border: 1px solid #e8c98a; border-left: 4px solid #c47a1b;
  border-radius: 0 8px 8px 0; padding: .9rem 1.1rem; margin-bottom: 1.25rem;
}
.conflicto strong { color: #96590c; display: block; margin-bottom: .25rem; }
.conflicto p { margin: .2rem 0 0; font-size: .88rem; color: #7a4e0d; }

/* ----- panel ----- */
.cifras {
  display: grid; gap: .9rem; margin-bottom: 1.25rem;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
}
.cifra {
  background: #fff; border: 1px solid #dfe6ec; border-left: 4px solid #94a3b8;
  border-radius: 10px; padding: 1rem 1.1rem; display: flex; flex-direction: column
}
.cifra--bien { border-left-color: #2f8f66; }
.cifra--aviso { border-left-color: #c47a1b; }
.cifra--malo { border-left-color: #c0392b; }
.cifra .valor { font-size: 1.9rem; font-weight: 800; line-height: 1.1; }
.cifra .rotulo {
  font-size: .78rem; text-transform: uppercase;
  letter-spacing: .05em; color: #64748b;
}

/* Barra de uso para el panel */

```

```

.riel { background: #eef2f6; border-radius: 999px; height: .65rem; overflow: hidde
.riel i {
  display: block; height: 100%; border-radius: 999px;
  background: linear-gradient(90deg, #42b883, #2f8f66);
}
.riel--lleno i { background: linear-gradient(90deg, #e8a33d, #c0392b); }

.filtros {
  display: flex; gap: .6rem; flex-wrap: wrap; margin-bottom: 1.1rem;
  padding: .85rem; background: #fff; border: 1px solid #dfe6ec; border-radius: 10p
}
.filtros input, .filtros select { font-size: .86rem; padding: .45rem .65rem; }
.filtros input[type="search"] { flex: 1; min-width: 190px; }

@media (max-width: 640px) {
  .fila { grid-template-columns: 1fr; }
}

```

6 Mapa de clases — qué clase va en qué elemento

CLASE	VA EN	PARA QUÉ
<code>.barra</code> · <code>.menu</code>	<code><header></code> y <code><nav></code> de <code>App.vue</code>	La navegación superior
<code>.contenido</code>	<code><main></code> que envuelve el <code><router-view></code>	Centra y limita el ancho
<code>.cabecera</code> · <code>.sub</code>	<code><header></code> de cada vista	Título a la izquierda, botón a la derecha
<code>.grilla</code> + <code>.tarjeta</code>	Contenedor y cada tarjeta	Rejilla que se acomoda sola
<code>.tabla-caja</code>	<code><div></code> que envuelve la <code><table></code>	Borde, esquinas y scroll horizontal
<code>.chip</code> + <code>.chip--pendiente</code> · <code>--entregado</code> · <code>--devuelto</code>	<code></code>	La píldora de estado. Hay una por estado
<code>.chip--atraso</code>	<code></code>	El aviso rojo de atraso

CLASE	VA EN	PARA QUÉ
<code>.ocupada</code>	<code><tr></code>	Apaga la fila que espera al servidor
<code>.atrasada</code>	<code><tr></code>	Fondo rojizo en las filas vencidas
<code>.filtros</code>	<code><div></code> con los inputs y selects	La barra de filtros
<code>.form</code> · <code>.fila</code>	<code><form></code> y grupos de dos campos	Una columna; <code>.fila</code> pone dos lado a lado
<code>.malo</code> · <code>.malo-txt</code>	El <code><input></code> y el <code><small></code> de abajo	El campo que el 422 marcó
<code>.conflicto</code>	<code><div></code> arriba del formulario	La caja del 409 . Separada a propósito
<code>.cifras</code> + <code>.cifra</code>	Rejilla y tarjeta del panel	Los números grandes
<code>.cifra--bien</code> · <code>--aviso</code> · <code>--malo</code>	Modificador de <code>.cifra</code>	El color del borde izquierdo
<code>.riel</code> + <code><i></code> dentro	Barra de progreso	El ancho del <code><i></code> se pone por <code>style</code>
<code>.riel--lleno</code>	Modificador de <code>.riel</code>	Pinta la barra en rojo
<code>.aviso</code> · <code>.aviso--error</code>	<code><div></code> centrado	Estados de carga, vacío y error
<code>.btn</code> · <code>--gris</code> · <code>--fino</code>	Botones y <code><router-link></code>	Funciona igual en los dos
<code>.mono</code>	<code><td></code> del código	Tipografía monoespaciada

CLAVE

Cómo usar las maquetas que vienen más abajo.

En cada requisito hay una **maqueta estática**: HTML plano, con datos escritos a mano y las clases ya puestas. Puedes abrirla en el navegador tal cual y ver cómo queda.

Tu trabajo es **convertirla en un componente Vue**: decidir qué se repite, qué aparece sólo a veces, qué clase depende de un dato, qué botón despacha qué. Esa conversión es el ejercicio. La maqueta te ahorra el CSS, no el pensamiento.

Los datos que aparecen escritos ("PR-0004", "Camila Rojas") son de muestra. Ninguno debe quedar escrito a mano en tu entrega.

CIERRE

Antes de pasar al Requisito 1

- ☐ El backend responde en el 3001
- ☐ El proyecto Vue levanta en el 5173
- ☐ `vuex`, `vue-router` y `axios` instalados
- ☐ Disparaste las cinco peticiones y **viste con tus ojos** qué devuelve cada una
- ☐ Sabes por qué la quinta responde 409 aunque todos los campos son válidos
- ☐ `src/estilos.css` creado e importado

1 Requisito 1 — Las cuatro pantallas y su navegación

Entregable: `src/router/index.js`, `src/App.vue` y cuatro vistas vacías. · Sin Vuex todavía.

1 Qué se necesita

La aplicación del mesón tiene cuatro pantallas. Antes de traer un solo dato, tienen que estar navegables:

RUTA	NOMBRE	VISTA	PARA QUÉ
<code>/</code>	<code>panel</code>	<code>PanelView.vue</code>	Cifras del pañol
<code>/prestamos</code>	<code>prestamos</code>	<code>PrestamosView.vue</code>	Planilla con filtros
<code>/prestamos/nuevo</code>	<code>prestamo-nuevo</code>	<code>PrestamoFormView.vue</code>	Registrar
<code>/prestamos/:id</code>	<code>prestamo-editar</code>	<code>PrestamoFormView.vue</code>	Editar
<code>/inventario</code>	<code>inventario</code>	<code>InventarioView.vue</code>	Catálogo de equipos

2 Criterios de aceptación

CIERRE

Se revisa esto, uno por uno

- ☐ Existe `src/router/index.js` y está enchufado en `main.js` con `.use(router)`
- ☐ Las cinco rutas tienen `name`. Ningún `router-link` usa `to="/ruta"`: todos usan `:to="{ name: '...' }"`
- ☐ `/prestamos/nuevo` está declarada **antes** que `/prestamos/:id`
- ☐ `/prestamos/:id` usa `props: true` y el componente recibe `id` como **prop**, no con `useRoute()`
- ☐ `PrestamoFormView.vue` se carga con `() => import(...)`. En la pestaña Network se ve que su JS llega en un archivo aparte y sólo al entrar a esa ruta

- ☐ Hay una ruta comodín que redirige al panel
- ☐ `App.vue` tiene la barra de navegación y un `<router-view />`, y **ninguna lógica de negocio**
- ☐ El enlace de la sección activa se marca solo, **sin ningún** `ref` **que lleve la cuenta**
- ☐ Navegar entre pantallas **no recarga la página**

CUIDADO

El error que te va a pasar, y te lo estoy avisando ahora.

Si declaras `/prestamos/:id` antes que `/prestamos/nuevo`, al entrar al formulario de creación el router va a capturar el texto `"nuevo"` como si fuera un id. Vas a pedirle al servidor el préstamo `"nuevo"`, vas a recibir un **404** y vas a revisar el backend durante veinte minutos buscando un problema que está en tu router.

La regla: **lo específico antes que lo genérico.**

MATIZ

Pista sobre la clase del enlace activo.

Vue Router le pone solo la clase `router-link-active` al enlace de la ruta en la que estás. Ya está en la hoja de estilos que copiaste. Si escribiste un `ref` llamado `pestanasActivas`, bórralo: estás replicando información que el router ya tiene.

3 La maqueta de la pantalla

Éste es el marco de la aplicación en HTML plano. Cópialo dentro del `<template>` de `App.vue` y adáptalo.

maqueta-layout.html

```
<header class="barra">
  <h1>📦 El Pañol</h1>

  <nav class="menu">
    <a href="#">Panel</a>
    <a href="#">Préstamos</a>
    <a href="#">Inventario</a>
```

```
</nav>

<a href="#" class="btn">+ Registrar préstamo</a>
</header>

<main class="contenido">
  <!-- Aquí se pinta la vista de la ruta actual -->
</main>
```

MATIZ

Lo que tienes que resolver en esta maqueta.

Esos cuatro `<a href="#">` **no pueden quedar así**: un `<a>` normal recarga la página entera y mata la SPA. Averigua con qué se reemplazan y cómo se apunta a una ruta *por su nombre*.

El `<main>` está vacío a propósito. Falta el elemento donde el router pinta la vista.

Y la clase que marca el enlace activo **no la escribes tú**: ya está en `estilos.css` y alguien te la va a poner sola. Averigua quién.

2 Requisito 2 — El inventario: tu primer módulo

Entregable: `src/api/http.js`, `src/store/index.js`, `src/store/modules/equipos.js` y `InventarioView.vue`.

1 Qué se necesita

La pantalla `/inventario` muestra todos los equipos en tarjetas. Cada tarjeta indica el nombre, la marca, la categoría, y —lo importante— **cuántas unidades hay disponibles de cuántas existen**. Los equipos en mantención se ven apagados y con una etiqueta.

Ahora, la pregunta que define este requisito: **¿quién más necesita esta lista?**

PANTALLA	PARA QUÉ NECESITA LOS EQUIPOS
<code>/inventario</code>	Mostrar las tarjetas
<code>/prestamos/nuevo</code>	Poblar el <code><select></code> y avisar cuántas quedan
<code>/prestamos</code>	Filtrar la planilla por equipo
<code>/</code>	Mostrar el uso del inventario

CLAVE

Cuatro pantallas, el mismo dato.

Ahí es cuando un dato deja de pertenecer al componente y pasa a pertenecer a la aplicación. Y lo que pertenece a la aplicación vive en el store. Ése es el criterio, no "es una lista, la pongo en Vuex".

2 Criterios de aceptación

CIERRE

La instancia de axios

- ☐ Existe `src/api/http.js` con **una sola** instancia creada con `axios.create()`
- ☐ La URL del backend aparece **exactamente una vez** en todo el proyecto
- ☐ Hay un interceptor de respuesta que normaliza los errores. Todas tus actions deben recibir siempre la misma forma, algo como `{ estado, mensaje, errores, stock }`
- ☐ El interceptor distingue "el servidor respondió con error" de "nunca hubo respuesta". Apaga el backend y comprueba que el mensaje es distinto

CIERRE

El store y el módulo

- ☐ `src/store/index.js` **sólo ensambla módulos**. No tiene state, ni mutations, ni datos propios
- ☐ Tiene `strict: import.meta.env.DEV`
- ☐ El módulo `equipos` tiene `namespaced: true`
- ☐ `state` es una **función** que devuelve un objeto, no un objeto
- ☐ El state guarda al menos: la lista, un indicador de carga y un mensaje de error
- ☐ **axios aparece únicamente dentro de una action**. Ni un import de axios en un componente
- ☐ Ninguna mutation tiene un `await`
- ☐ Ninguna action asigna directamente al state: todas usan `commit`
- ☐ La action **no vuelve a pedir el catálogo si ya lo tiene**, salvo que se le pida forzar
- ☐ Hay al menos tres getters. Uno de ellos debe devolver un **índice** `id → equipo` para no andar haciendo `.find()` en cada fila de una tabla

CIERRE

La vista

- ☐ Maneja **cuatro** estados: cargando, error, vacío y con datos. No dos
- ☐ El estado vacío no dice "cargando..." para siempre
- ☐ Cuando hay error hay un botón para reintentar
- ☐ Hay un botón "Recargar" que fuerza la petición

- ☐ Ir a otra pantalla y volver **no dispara una segunda petición**. Compruébalo en la pestaña Network

MATIZ

Sobre el namespace, que es donde todos se caen.

El `state` se anida como objeto y los `getters`, `actions` y `mutations` se registran con una clave de texto. La sintaxis no es simétrica y confunde el primer día:

`store.state.equipo.lista` lleva **punto**.

`store.getters['equipos/disponibles']` lleva **barra y corchetes**.

Y ojo: `dispatch('cargar')` sin el prefijo **no hace nada y no avisa**. Vuex busca una action global con ese nombre, no la encuentra y se queda callado. Tu pantalla queda vacía sin un solo error en consola.

3 La maqueta de la pantalla

Dos tarjetas de muestra, con las clases puestas. Una de un equipo con unidades libres y otra de uno en mantención, para que veas los dos casos.

■ maqueta-inventario.html

```
<section>
  <header class="cabecera">
    <div>
      <h2>Inventario</h2>
      <p class="sub">8 equipos · 6 con unidades disponibles</p>
    </div>
    <button class="btn">Recargar</button>
  </header>

  <div class="grilla">

    <article class="tarjeta">
      <h3>Notebook Lenovo ThinkPad</h3>
      <p class="sub">Lenovo · Computación</p>
```

```

    <p class="sub" style="margin-top:.75rem">
      <strong>3</strong> de 12 disponibles
    </p>
    <span class="riel">
      <i style="width: 75%"></i>
    </span>

    <p class="sub" style="margin-top:.75rem">$620.000 por unidad</p>
  </article>

  <article class="tarjeta" style="opacity:.6">
    <h3>Osciloscopio</h3>
    <p class="sub">Rigol · Medición</p>

    <p class="sub" style="margin-top:.75rem">
      <strong>2</strong> de 2 disponibles
    </p>
    <span class="riel riel--lleno">
      <i style="width: 0%"></i>
    </span>

    <p class="sub" style="margin-top:.75rem">$540.000 por unidad</p>

    <p style="margin:.5rem 0 0">
      <span class="chip chip--atraso">En mantención</span>
    </p>
  </article>

</div>
</section>

```

Y las tres cajas de aviso, para los estados que no son "hay datos":

■ maqueta-avisos.html

```

<div class="aviso">Consultando el inventario...</div>

<div class="aviso aviso--error">
  No se pudo contactar al servidor.
  <button class="btn btn--fino">Reintentar</button>
</div>

```

```
<div class="aviso">No hay equipos registrados.</div>
```

MATIZ

Lo que tienes que resolver en esta maqueta.

- Hay **dos** tarjetas escritas a mano y en la realidad son ocho. ¿Qué directiva las genera? ¿Qué le tienes que pasar además del dato?
- El `style="width: 75%"` del riel está calculado a mano. En tu versión ese número sale de dos campos del equipo. Piensa cuáles y cómo se enlaza un estilo.
- La segunda tarjeta lleva `opacity:.6`, la clase `riel--lleno` y una píldora. Las tres cosas dependen de **condiciones distintas**: una del campo `operativo` y las otras de `disponibles`. Averigua cómo se pone una clase sólo cuando se cumple algo.
- Las tres cajas de aviso y la grilla son **excluyentes**: nunca se ven dos a la vez. ¿Con qué familia de directivas se encadenan?
- El texto "6 con unidades disponibles" no debe calcularse en el template.

CLAVE

Una ayuda que sí te doy, porque es presentación y no estado.

Para el precio y el nombre legible de la categoría, mete esto en tu `<script setup>`:

✓ Ayudantes de presentación – cópialos **SOLUCIÓN**

```
const pesos = (v) =>
  v.toLocaleString('es-CL', { style: 'currency', currency: 'CLP', maximumFractionD

const etiquetas = {
  computacion: 'Computación',
  audiovisual: 'Audiovisual',
  redes: 'Redes',
  medicion: 'Medición',
```

```
mobiliario: 'Mobiliario'
```

```
}
```


3 Requisito 3 — La planilla: tu segundo módulo

Entregable: `src/store/modules/prestamos.js` y `PrestamosView.vue`.

1 Qué se necesita

La pantalla `/prestamos` es la herramienta del mesón. Muestra una tabla paginada con todos los préstamos y deja hacer el trabajo del día:

ACCIÓN	CÓMO SE HACE CONTRA LA API
Marcar que retiraron el equipo	<code>PATCH { "estado": "entregado" }</code>
Marcar que lo devolvieron	<code>PATCH { "estado": "devuelto" }</code>
Editar un préstamo	Navegar a <code>/prestamos/:id</code>
Borrar un registro	<code>DELETE</code> , con confirmación previa

Y filtrar por: texto libre, estado, equipo, categoría y **sólo atrasados**.

EN VIVO

¿Por qué un módulo nuevo y no meter esto en equipos?

Porque son **dos ciclos de vida distintos**. El inventario se pide una vez y se queda quieto. Los préstamos cambian a cada rato: se crean, se entregan, se devuelven, se filtran, se pagan. Tienen su propio estado de carga, su propio error, sus propios filtros.

Meterlos juntos los obligaría a compartir un `cargando` que significa cosas distintas para cada uno. **Dos ciclos de vida, dos módulos**. Ésa es la regla, no el tamaño del archivo.

2 Criterios de aceptación

El módulo

- ☐ `namespaced: true` y `state` como función
- ☐ **Los filtros viven en el state**, no en `ref` del componente. Se comprueba así: filtra por "atrasados", ándate al panel, vuelve — **el filtro sigue puesto**
- ☐ La `meta` de paginación se guarda **tal como llega del servidor**. No la recalculas
- ☐ Cambiar cualquier filtro **vuelve a la página 1**
- ☐ Los filtros vacíos **no se envían** como parámetro. Mandar `estado=""` te va a devolver un `400`
- ☐ Entregar o devolver **no recarga la lista completa**. El servidor ya te devolvió el préstamo actualizado en la respuesta del PATCH: úsalo para reemplazar esa fila
- ☐ Hay un estado que indica **qué fila** está esperando al servidor, y se deshabilita sólo esa fila, no la tabla entera
- ☐ Ese indicador se limpia también cuando la petición **falla**. Pista: `finally`
- ☐ Si borras el último registro de una página, la vista **no queda en blanco**

La tabla

- ☐ Muestra código, solicitante con su área, equipo, cantidad, fechas y estado
- ☐ Las filas atrasadas se distinguen visualmente y muestran **cuántos días** llevan
- ☐ Usas `equipoNombre` y `atrasado` que ya vienen resueltos. **No los recalculas en el cliente**
- ☐ Los botones de acción aparecen según el estado: no ofrezcas "Devolver" en algo que ya está devuelto
- ☐ El `DELETE` pide confirmación
- ☐ Hay paginación con Anterior y Siguiente, deshabilitados en los extremos
- ☐ El selector de filtro por equipo se alimenta del **módulo** `equipos`
- ☐ Estado vacío distinto según haya filtros puestos o no: "nada calza con esos filtros" no es lo mismo que "todavía no hay préstamos"

CUIDADO

No uses `v-model` sobre los filtros del store.

`v-model` escribe directo sobre lo que le des. Contra el state de Vuex eso es exactamente lo que `strict` prohíbe, y te vas a llevar un *"Do not mutate vuex store state outside mutation handlers"* en la consola.

Se lee con `:value` y se escribe con `@input` o `@change` despachando la action. Sí, es más largo.

MATIZ

Cuándo un módulo necesita al otro, y cuándo no.

La **tabla** no necesita el módulo `equipos`: el backend ya manda `equipoNombre` resuelto en cada fila. Úsalo y no cruces nada.

El **selector del filtro** sí lo necesita, porque hacen falta *todos* los equipos, no sólo los que aparecen en la página actual.

La forma más limpia de comunicar dos módulos es **no comunicarlos**. La segunda es que el componente lea de los dos. Sólo si eso no alcanza se usa `rootGetters` o `dispatch(nombre, null, { root: true })` —y ojo con ese `null`, que es el payload: si lo omites, Vuex interpreta `{ root: true }` como el payload y la llamada no hace nada. Sin error.

3 La maqueta de la pantalla

Tres filas de muestra: una pendiente, una entregada y atrasada, y una devuelta. Fíjate en que cada una tiene un juego de botones distinto.

maqueta-planilla.html

```
<section>
  <header class="cabecera">
    <div>
      <h2>Préstamos</h2>
      <p class="sub">Mostrando 6 de 15</p>
    </div>
    <a href="#" class="btn">+ Registrar</a>
```

```

</header>

<div class="filtros">
  <input type="search" placeholder="Buscar por solicitante, área o código...">

  <select>
    <option value="">Todos los estados</option>
    <option value="pendiente">Pendientes</option>
    <option value="entregado">Entregados</option>
    <option value="devuelto">Devueltos</option>
  </select>

  <select>
    <option value="">Todos los equipos</option>
    <option>Notebook Lenovo ThinkPad</option>
  </select>

  <select>
    <option value="">Atrasados y al día</option>
    <option value="true">Sólo atrasados</option>
    <option value="false">Sólo al día</option>
  </select>

  <button class="btn btn--gris">Limpiar</button>
</div>

<div class="tabla-caja">
  <table>
    <thead>
      <tr>
        <th>Código</th><th>Solicitante</th><th>Equipo</th>
        <th>Cant.</th><th>Devolución</th><th>Estado</th><th></th>
      </tr>
    </thead>
    <tbody>

      <tr>
        <td class="mono">PR-0003</td>
        <td>
          <strong>Andrea Bouffanais</strong>
          <small>Charla de Titulación</small>
        </td>
        <td>Proyector portátil</td>
        <td>2</td>
        <td>7 ago</td>
      </tr>
    </tbody>
  </table>
</div>

```

```
<td><span class="chip chip--pendiente">pendiente</span></td>
<td style="white-space:nowrap">
  <button class="btn btn--fino">Entregar</button>
  <a href="#" class="btn btn--fino btn--gris">Editar</a>
  <button class="btn btn--fino btn--gris">Eliminar</button>
</td>
</tr>

<tr class="atrasada">
  <td class="mono">PR-0004</td>
  <td>
    <strong>Marcelo Vera</strong>
    <small>Registro Audiovisual</small>
  </td>
  <td>Cámara de video</td>
  <td>2</td>
  <td>
    31 jul
    <small>4 días de atraso</small>
  </td>
  <td>
    <span class="chip chip--entregado">entregado</span>
    <span class="chip chip--atraso">Atrasado</span>
  </td>
  <td style="white-space:nowrap">
    <button class="btn btn--fino">Devolver</button>
    <a href="#" class="btn btn--fino btn--gris">Editar</a>
    <button class="btn btn--fino btn--gris">Eliminar</button>
  </td>
</tr>

<tr>
  <td class="mono">PR-0006</td>
  <td>
    <strong>Ignacio Soto</strong>
    <small>Laboratorio de Redes</small>
  </td>
  <td>Router configurable</td>
  <td>6</td>
  <td>29 jul</td>
  <td><span class="chip chip--devuelto">devuelto</span></td>
  <td style="white-space:nowrap">
    <button class="btn btn--fino btn--gris">Eliminar</button>
  </td>
</tr>
```

```

        </tbody>
    </table>
</div>

<nav class="cabecera" style="margin-top:1rem;justify-content:center;gap:1rem">
    <button class="btn btn--gris">← Anterior</button>
    <span class="sub">Página 1 de 3</span>
    <button class="btn btn--gris">Siguiete →</button>
</nav>
</section>

```

MATIZ

Lo que tienes que resolver en esta maqueta.

- Las tres filas son **una sola** repetida. Compáralas para descubrir qué cambia entre ellas.
- Las píldoras de estado son tres clases distintas para tres valores distintos. **No escribas tres bloques condicionales**: mira los nombres de las clases y el valor del campo `estado`. Se pueden armar en una línea.
- La segunda fila tiene `class="atrasada"` y un `<small>` con los días. Las dos cosas dependen del mismo campo.
- Los botones cambian por fila. Escribe cuál aparece con cada estado antes de programar:
 - `pendiente` → Entregar, Editar, Eliminar
 - `entregado` → Devolver, Editar, Eliminar
 - `devuelto` → sólo Eliminar
- Falta la clase `ocupada`: ninguna fila de la maqueta la tiene, porque sólo aparece mientras esa fila espera al servidor. Tú tienes que agregarla.
- Los cuatro controles de filtro no guardan nada. **Ojo acá**: el atajo obvio para enlazarlos es justamente el que `strict` te va a rechazar. Vuelve a leer el recuadro de más arriba.
- Los botones de paginación deben deshabilitarse en los extremos. El dato para saberlo viene en `meta`.
- Los `<a href="#">` tienen el mismo problema que en el Requisito 1.

CLAVE

Ayudante de presentación — cópialo.

✓ Formato de fecha corto **SOLUCIÓN**

```
const fecha = (iso) =>  
  new Date(iso + 'T00:00:00').toLocaleDateString('es-CL', { day: 'numeric', month:
```

4 Requisito 4 — El formulario en Vuex

Entregable: el bloque de formulario dentro de `prestamos.js` y `PrestamoFormView.vue`. · **Es el requisito que más pesa en la revisión.**

1 Qué se necesita

Un formulario que sirve para **dos cosas**: registrar un préstamo nuevo (`/prestamos/nuevo`) y editar uno existente (`/prestamos/:id`). Mismo componente, misma vista, distinto comportamiento según haya `id` o no.

CAMPO	CONTROL	DE DÓNDE SALEN LAS OPCIONES
<code>equipoId</code>	<code><select></code>	Módulo <code>equipos</code>
<code>solicitante</code>	texto	—
<code>area</code>	texto	—
<code>cantidad</code>	número	—
<code>fechaRetiro</code>	<code>date</code>	—
<code>fechaDevolucion</code>	<code>date</code>	—
<code>observacion</code>	<code>textarea</code>	—

EN VIVO

"Pero un formulario es estado local, eso no va en Vuex."

En general, cierto. Un campo de búsqueda, un modal abierto o cerrado: eso es del componente. Pero cuenta de dónde viene lo que necesita *este* formulario: las opciones del selector vienen del módulo `equipos` ; los errores vienen de la respuesta del `POST` ; el préstamo a editar viene del servidor; y la planilla necesita enterarse de que se creó uno nuevo.

Nada de eso es local. Ése es el criterio: si puedes enumerar razones concretas, va al store. Si no puedes, déjalo local.

2

Lo central: 422 y 409 no se muestran igual

⇒ 422 – un campo está mal

```
{
  "message": "Algunos campos
              tienen problemas.",
  "errores": {
    "solicitante": "mínimo 3
                  caracteres",
    "cantidad": "entero entre
                1 y 20",
    "fechaDevolucion": "no puede
                       ser anterior al retiro"
  }
}
```

Hay un **campo culpable**. El mensaje va **debajo de ese input** y el usuario lo arregla corrigiéndolo.

⇒ 409 – no alcanzan las unidades

```
{
  "message": "No hay unidades
              suficientes de Notebook
              Lenovo ThinkPad. Pediste 5
              y sólo quedan 3.",
  "stock": {
    "equipo": "Notebook...",
    "solicitadas": 5,
    "disponibles": 3,
    "comprometidas": 9,
    "stockTotal": 12
  }
}
```

Ningún campo está mal. Ponerlo debajo de un input sería mentirle al usuario. Va arriba, en su propia caja.

CUIDADO

Si guardas los dos en la misma variable del state, este requisito está mal resuelto.

Son dos problemas de naturaleza distinta y necesitan dos lugares distintos en el state y dos tratamientos distintos en la pantalla. Es el atajo más común y el que produce interfaces que confunden a quien las usa.

3 Criterios de aceptación

CIERRE

El estado del formulario

- ☐ Los campos del formulario están en el **state del módulo**, no en `ref` del componente
- ☐ El formulario en blanco se genera con una **función**, no con una constante. Compruébalo: crea un préstamo, vuelve a "Nuevo" y verifica que **no arrastra los datos del anterior**
- ☐ Escribir en un campo dispara una **mutation**. En Devtools se ve una mutation con nombre y valor por cada tecla
- ☐ Los errores de campo (`422`) y el conflicto de stock (`409`) están en **dos propiedades separadas** del state
- ☐ Hay un indicador de "guardando" que deshabilita el botón
- ☐ Los estados y categorías del formulario salen de `/prestamos/opciones`, **no escritos a mano** en el template
- ☐ No hay **ni un** `v-model` apuntando al store

CIERRE

El comportamiento

- ☐ Entrar a `/prestamos/nuevo` deja el formulario limpio

- ☐ Entrar a `/prestamos/7` lo llena con los datos de ese préstamo, **pedidos frescos al servidor** (no tomados de la lista que tenías cargada, que puede estar vieja)
- ☐ Entrar a `/prestamos/9999` no deja al usuario mirando un formulario vacío para siempre
- ☐ El título dice "Registrar préstamo" o "Editar préstamo" según corresponda
- ☐ Los mensajes del `422` se muestran **tal como los manda el servidor**. No los traduzcas ni los reescribas
- ☐ El input con error se marca visualmente, con la clase `.malo` de la hoja de estilos
- ☐ Corregir **un** campo borra **sólo su** mensaje. Los demás se quedan
- ☐ El `409` se muestra en la caja `.conflicto`, arriba, e incluye **cuántas unidades hay disponibles** — ese número te lo da el servidor, úsalo
- ☐ Al guardar bien: se limpia el formulario, se refresca la planilla y **recién ahí** se navega con `router.push`
- ☐ Al guardar mal: **no se navega**. El usuario tiene que ver qué pasó
- ☐ La action de guardar **devuelve un resultado** (algo como `{ ok, tipo }`) en vez de lanzar una excepción, para que el componente decida si navegar

MATIZ

Cómo probar el 409 sin preparar nada.

Elige **Cámara de video**: existen 3 y las 3 están fuera. Cualquier cantidad te va a dar 409 de inmediato.

Para ver el mensaje con números intermedios, elige **Notebook Lenovo**: hay 12, hay 9 comprometidos, quedan 3. Pide 5 y el servidor te va a decir exactamente eso.

Y después, para ver el ciclo completo: en la planilla marca como *devuelto* el préstamo PR-0004 (2 cámaras). Vuelve al formulario y pide una cámara. **Ahora entra**. Devolver libera el stock.

CLAVE

Una pista sobre el selector, que sube mucho la calidad de tu entrega.

Como ya tienes el módulo `equipos` cargado, puedes mostrar **cuántas unidades quedan al lado de cada opción** del `<select>`, y avisar antes de enviar si la

cantidad pedida supera lo disponible.

Eso es validación de **cortesía**: hace la interfaz más amable, evita un viaje al servidor. Pero ojo con la conclusión equivocada — **no reemplaza al 409**. Entre que tu pantalla leyó "quedan 3" y que el usuario aprieta Guardar, alguien más pudo llevarse las tres. Por eso el 409 tiene que seguir manejado.

4

La maqueta de la pantalla

El formulario con **los dos tipos de error visibles a la vez**, para que veas dónde va cada uno. En la aplicación real casi nunca aparecerán juntos.

■ maqueta-formulario.html

```
<section class="tarjeta" style="max-width:640px;margin:0 auto">
  <h2>Registrar préstamo</h2>

  <!-- ===== EL 409: arriba, en su propia caja ===== -->
  <div class="conflicto">
    <strong>No alcanzan las unidades</strong>
    <p>No hay unidades suficientes de Notebook Lenovo ThinkPad. Pediste 5 y sólo q
    <p>De 12 unidades de <strong>Notebook Lenovo ThinkPad</strong>, hay 9 fuera de
  </div>

  <form class="form">
    <div class="fila">
      <label>
        <span>Equipo</span>
        <select>
          <option value="">Elige un equipo...</option>
          <option>Notebook Lenovo ThinkPad – quedan 3</option>
          <option>Proyector portátil – quedan 2</option>
        </select>
      </label>

      <label>
        <span>Cantidad</span>
        <!-- ===== EL 422: el campo se marca y el mensaje va DEBAJO ===== -->
        <input type="number" min="1" max="20" value="99" class="malo">
        <small class="malo-txt">La cantidad debe ser un número entero entre 1 y 20
      </label>
```

```

</div>

<label>
  <span>Quién retira</span>
  <input type="text" placeholder="Nombre de la persona o del curso">
</label>

<label>
  <span>Área o asignatura</span>
  <input type="text" placeholder="Taller de Redes, Certificación G7...">
</label>

<div class="fila">
  <label>
    <span>Fecha de retiro</span>
    <input type="date">
  </label>

  <label>
    <span>Fecha de devolución</span>
    <input type="date" class="malo">
    <small class="malo-txt">La devolución no puede ser anterior al retiro.</small>
  </label>
</div>

<label>
  <span>Observación <em style="font-weight:400;color:#94a3b8">(opcional)</em><
  <textarea rows="2" maxlength="200"></textarea>
</label>

<footer class="cabecera" style="margin:0;justify-content:flex-end;gap:.6rem">
  <button type="button" class="btn btn--gris">Cancelar</button>
  <button type="submit" class="btn">Registrar</button>
</footer>
</form>
</section>

```

CUIDADO

Mira dónde está cada error en esa maqueta. Ésa es la lección completa del requisito.

El **422** aparece *dos veces*, y las dos debajo de un campo: la clase `malo` en el input y un `<small class="malo-txt">` con el mensaje del servidor. Hay un campo

culpable y el usuario lo corrige ahí.

El **409** aparece *una vez*, arriba, en un `div.conflicto`. Ningún campo está mal. Ponerlo debajo de un input sería mentirle al usuario.

En tu store esos dos tienen que salir de **dos propiedades distintas del state**. Si salen de la misma, el requisito está mal resuelto aunque la pantalla se vea igual.

MATIZ

Lo que tienes que resolver en esta maqueta.

- La caja del `conflicto` y los dos `<small class="malo-txt">` están escritos fijos. En tu versión aparecen **sólo cuando existe el error correspondiente**.
- La clase `malo` del input también es condicional, y depende de **si hay mensaje para ese campo en particular**.
- Ninguno de los siete campos está enlazado al store. Ojo con el atajo obvio: `strict` lo rechaza. Ya sabes cuál es.
- Las dos `<option>` de equipos están a mano, y el "quedan 3" también. Los dos salen del módulo `equipos`.
- El título dice "Registrar préstamo" fijo. Tiene que cambiar cuando estés editando.
- El `<form>` no hace nada al enviarse, y el navegador va a recargar la página. Averigua qué modificador de evento lo impide.
- El botón de enviar debe deshabilitarse mientras se guarda, y decir otra cosa.
- Falta lo más importante y no se ve en la maqueta: **al corregir un campo, su mensaje tiene que desaparecer — y sólo el suyo**.

5 Requisito 5 — El panel del encargado

Entregable: un tercer módulo para el resumen y `PanelView.vue`.

1 Qué se necesita

La pantalla `/` es para el encargado del pañol. Necesita cuatro cifras grandes y dos bloques de detalle:

ELEMENTO	QUÉ MUESTRA
Cifra 1	Total de préstamos registrados
Cifra 2	Unidades fuera del pañol ahora mismo
Cifra 3	Atrasados — en rojo si hay alguno
Cifra 4	Valor en circulación, formateado en pesos
Bloque A	Uso del inventario por equipo, con barra de porcentaje
Bloque B	Préstamos vivos por categoría

2 La decisión de este requisito

CUIDADO

Tienes dos formas de saber cuántos préstamos hay entregados. Sólo una es correcta acá.

A. Un getter sobre `state.prestamos.lista`. Instantáneo, gratis, no falla. Pero cuenta **los 6 de la página actual**, no los 15 del pañol.

B. Pedir `/prestamos/resumen`. Cuesta una petición y puede fallar, pero ve **toda la base de datos**.

Si usas el getter para el panel, tu pantalla va a mostrar un número con toda seguridad y va a estar equivocado. Eso es peor que un error: un error se ve, un

número equivocado no.

No le preguntes a la red lo que ya tienes en memoria, y no inventes con tu memoria lo que sólo sabe la base de datos.

3

Criterios de aceptación

CIERRE

El módulo del resumen

- ☐ Es un **tercer módulo**, separado de `prestamos`. Justificación: si el resumen falla, la planilla tiene que seguir funcionando
- ☐ Todas las cifras grandes salen de `/prestamos/resumen`, **ninguna de un getter sobre la página cargada**
- ☐ Tiene caché: no vuelve a pedir el resumen si lo pidió hace menos de 30 segundos
- ☐ Hay un botón que **fuerza** la actualización saltándose el caché
- ☐ Hay al menos dos getters que dan **forma** a lo que ya trajo el servidor. Eso sí es legítimo: no estás inventando totales, estás preparando datos completos para pintarlos

CIERRE

El panel

- ☐ La cifra de atrasados usa la clase `.cifra--malo` cuando hay al menos uno
- ☐ El valor en circulación está formateado como moneda chilena. Pista:
`toLocaleString('es-CL', { style: 'currency', currency: 'CLP' })`
- ☐ Las barras de uso se pintan con `.riel`, y usan `.riel--lleno` cuando el equipo está al 100%
- ☐ Se maneja el estado de carga y el de error
- ☐ Hay una nota visible que aclara que las cifras son del pañol completo, no de la página que se ve en la planilla

La demostración que tienes que poder hacer al entregar.

Abre el panel, anota el total. Abre la planilla y mira el contador de la cabecera. **No coinciden.** Tienes que poder explicar por qué, y por qué *ninguno de los dos está mal*.

Después crea un préstamo y vuelve al panel: la cifra **no cambia**, porque el caché de 30 segundos todavía no vence. Aprieta Actualizar y ahí sí. Eso tampoco es un bug — es una decisión, y tienes que poder defenderla.

4

La maqueta de la pantalla

maqueta-panel.html

```
<section>
  <header class="cabecera">
    <h2>Panel del pañol</h2>
    <button class="btn">Actualizar</button>
  </header>

  <!-- Las cuatro cifras grandes -->
  <div class="cifras">
    <article class="cifra">
      <span class="valor">15</span>
      <span class="rotulo">Préstamos</span>
    </article>

    <article class="cifra cifra--bien">
      <span class="valor">32</span>
      <span class="rotulo">Unidades fuera</span>
    </article>

    <article class="cifra cifra--malo">
      <span class="valor">2</span>
      <span class="rotulo">Atrasados</span>
    </article>

    <article class="cifra cifra--aviso">
      <span class="valor">$8.965.000</span>
      <span class="rotulo">En circulación</span>
    </article>
  </div>
</section>
```

```
</article>
</div>

<div class="grilla" style="grid-template-columns:1fr 1fr">

  <article class="tarjeta">
    <h3>Uso del inventario</h3>
    <p class="sub">El más pedido es <strong>Multímetro digital</strong>.</p>

    <div style="margin-bottom:.7rem">
      <p class="sub" style="display:flex;justify-content:space-between;margin:0">
        <span>Notebook Lenovo ThinkPad</span>
        <strong>9 / 12</strong>
      </p>
      <span class="riel"><i style="width: 75%"></i></span>
    </div>

    <div style="margin-bottom:.7rem">
      <p class="sub" style="display:flex;justify-content:space-between;margin:0">
        <span>Cámara de video</span>
        <strong>3 / 3</strong>
      </p>
      <span class="riel riel--lleno"><i style="width: 100%"></i></span>
    </div>
  </article>

  <article class="tarjeta">
    <h3>Préstamos vivos por categoría</h3>
    <table>
      <thead>
        <tr><th>Categoría</th><th>Préstamos</th></tr>
      </thead>
      <tbody>
        <tr><td>computacion</td><td>3</td></tr>
        <tr><td>audiovisual</td><td>3</td></tr>
      </tbody>
    </table>

    <p class="sub" style="margin-top:1rem">
      Valor en circulación: <strong>$8.965.000</strong>
    </p>
  </article>

</div>
```

```
<p class="sub" style="text-align:center;margin-top:1rem">
  Cifras del servidor sobre los 15 préstamos del pañol,
  no sobre la página que estás viendo en la planilla.
</p>
</section>
```

MATIZ

Lo que tienes que resolver en esta maqueta.

- Las cuatro tarjetas de cifra tienen **modificadores distintos** (`cifra--bien` , `--malo` , `--aviso` , y la primera ninguno). Piensa si conviene repetir cuatro bloques o generar los cuatro desde una estructura de datos.
- El `cifra--malo` de "Atrasados" no debería ser fijo: si no hay ninguno atrasado, esa tarjeta no tiene por qué estar en rojo.
- Sólo hay dos barras de uso escritas y en la realidad son ocho.
- Los anchos (`75%` , `100%`) y la clase `riel--lleno` están puestos a mano. El resumen del servidor te trae un campo que resuelve el ancho directo — búscalo.
- La tabla de categorías recorre un **objeto**, no un arreglo. Averigua qué recibe la directiva de repetición cuando le pasas un objeto: **el orden de los dos parámetros sorprende a todo el mundo la primera vez**.
- Los montos vienen del servidor como números planos. El formato de moneda es cosa tuya.
- Faltan el estado de carga y el de error.
- Y lo esencial: **ninguno de estos números puede salir de un getter sobre la lista de préstamos que tengas cargada**. Ahí hay 6 registros; en el pañol hay 15.

1 Cómo se revisa tu entrega

Se abre tu proyecto y se hacen estas nueve comprobaciones, en este orden. Puedes hacértelas tú antes de entregar:

#	COMPROBACIÓN	CÓMO SE HACE
1	Ningún componente importa axios	Buscar <code>from 'axios'</code> en todo <code>src/</code> . Sólo puede aparecer en <code>api/http.js</code>
2	La URL del backend aparece una sola vez	Buscar <code>localhost:3001</code>
3	Ninguna mutation tiene <code>await</code>	Leer los bloques <code>mutations</code>
4	Todos los módulos tienen <code>namespaced: true</code> y <code>state</code> como función	Leer las tres primeras líneas de cada módulo
5	Los filtros sobreviven a la navegación	Filtrar, salir a otra pantalla, volver
6	Los errores del 422 se pintan bajo cada campo	Enviar el formulario con todos los campos malos
7	El 409 tiene tratamiento propio y dice cuántas quedan	Pedir 5 notebooks
8	El panel no miente	Comparar el total del panel con el contador de la planilla
9	Devolver libera stock	Devolver PR-0004 y pedir una cámara

CLAVE

Si algo no alcanzaste a terminar, dilo en el README de tu proyecto.

Una entrega incompleta que reconoce qué falta y por qué vale más que una que pretende estar completa. En el trabajo real pasa exactamente lo mismo.

SÍNTOMA	CAUSA	ARREGLO
La pantalla queda vacía y no hay ningún error	<code>dispatch('cargar')</code> sin el prefijo del módulo	<code>dispatch('equipos/cargar')</code>
<i>"Do not mutate vuex store state outside mutation handlers"</i>	<code>v-model</code> contra el store, o una action que asigna al state	<code>:value</code> + <code>@input</code> con <code>commit</code>
El formulario nuevo arrastra datos del anterior	El objeto por defecto es una constante	Que sea una función
El backend responde <code>400</code> al filtrar	Estás enviando filtros vacíos como <code>estado=""</code>	No incluirlos en <code>params</code> si están vacíos
El formulario de "nuevo" pide el préstamo <code>"nuevo"</code>	<code>/prestamos/:id</code> declarada antes	Lo específico antes que lo genérico
<code>{ root: true }</code> no hace nada	Falta el <code>null</code> del payload	<code>dispatch('otro/accion', null, { root: true })</code>
Subir la cantidad de un préstamo da <code>409</code> siempre	Es un problema del servidor, no tuyo — y ya está resuelto: al editar se excluye a sí mismo del cálculo	Si te pasa, revisa que estés mandando el <code>PUT</code> al id correcto
La fila queda deshabilitada para siempre	El indicador de "ocupada" no se limpia cuando falla	<code>finally</code>

Extras, en orden de dificultad

1. **Aviso de éxito.** Al crear un préstamo, que la planilla muestre una franja verde con el código del préstamo creado, que se desvanezca a los 4 segundos.
2. **Invalidar el resumen.** Que crear, editar o eliminar un préstamo fuerce la actualización del panel. Pista: `dispatch('resumen/cargar', { forzar: true }, { root: true })`.
3. **Disponibilidad viva en el formulario.** Que al elegir un equipo aparezca debajo "quedan N unidades", y que el campo cantidad limite su máximo a ese número. Recuerda: es cortesía, el 409 igual se maneja.
4. **Filtros en la URL.** Que los filtros de la planilla se reflejen como *query params* (`/prestamos?estado=entregado`) y que al recargar la página se restauren. Pista: `router.replace({ query })` y leer `route.query` al montar. Esto es Router y Vuex trabajando juntos, y es más difícil de lo que parece.
5. **Vista de detalle de un equipo.** Ruta `/inventario/:id` que muestre el equipo y **todos sus préstamos vivos**. Necesita los dos módulos a la vez.
6. **Migrar a Pinia.** Los mismos tres módulos, sin mutations y sin prefijos. Vas a descubrir que lo que costó entender —qué dato va dónde— se transfiere entero, y que la sintaxis era lo fácil.

EN VIVO

Lo que estás practicando de verdad.

No es Vuex. Vuex es la excusa. Lo que estás practicando es **decidir dónde va cada cosa**: qué dato pertenece a la aplicación y cuál al componente, dónde vive la llamada a la red, cuándo un cálculo local alcanza y cuándo hay que preguntarle al servidor, y por qué un conflicto de negocio no se muestra igual que un campo mal escrito.

Eso no cambia cuando cambies de librería, de framework o de lenguaje. La sintaxis se olvida en un año. Esto no.

